```
Start coding or generate with AI.
```

DAY 1 MONDAY

Double-click (or enter) to edit

```
def common_end(a, b):
  return a[0] == b[0] or a[-1] == b[-1]
```

```
def sum3(nums):
    return sum(nums)
```

```
def rotate_left3(nums):
    return [nums[1], nums[2], nums[0]]
```

```
def rotate_left3(nums):
    return [nums[1], nums[2], nums[0]]
```

```
def reverse3(nums):
    return [nums[2], nums[1], nums[0]]
```

```
def max_end3(nums):
    m = max(nums[0], nums[-1])
    return [m, m, m]
```

```
def sum2(nums):
    if len(nums) == 0:
        return 0
    if len(nums) == 1:
        return nums[0]
    return nums[0] + nums[1]
```

```
def middle_way(a, b):
    return [a[1], b[1]]
```

```
def make_ends(nums):
    return [nums[0], nums[-1]]
```

```
def has23(nums):
    return 2 in nums or 3 in nums
```

```
def count_evens(nums):
    count = 0
    for n in nums:
        if n % 2 == 0:
            count += 1
    return count
```

```
def big_diff(nums):
    return max(nums) - min(nums)
```

```
def string_times(str, n):
    return str * n
```

```
def front_times(str, n):
    front = str[:3]
```

```
    return front * n
```

Start coding or generate with AI.

Day 2 Tuesday

```
def front_times(str, n):
    front = str[:3]
    return front * n
```

```
def string_bits(str):
    result = ""
    for i in range(0, len(str), 2):
        result += str[i]
    return result
```

```
def string_splosion(str):
    result = ""
    for i in range(len(str)+1):
        result += str[:i]
    return result
```

```
def last2(str):
    if len(str) < 2:
        return 0
    last = str[-2:]
    count = 0
    for i in range(len(str)-2):
        if str[i:i+2] == last:
            count += 1
    return count
```

```
def array_count9(nums):
    count = 0
    for n in nums:
        if n == 9:
            count += 1
    return count
```

```
def array_front9(nums):
    for i in range(min(4, len(nums))):
        if nums[i] == 9:
            return True
    return False
```

```
def array123(nums):
    for i in range(len(nums)-2):
        if nums[i]==1 and nums[i+1]==2 and nums[i+2]==3:
            return True
    return False
```

```
def string_match(a, b):
    count = 0
    for i in range(min(len(a), len(b))-1):
        if a[i:i+2] == b[i:i+2]:
            count += 1
    return count
```

```
def double_char(str):
    result = ""
    for ch in str:
        result += ch*2
    return result
```

```
def centered_average(nums):
    nums.sort()
    return sum(nums[1:-1]) // (len(nums)-2)

def sum13(nums):
    total = 0
    skip = False
    for n in nums:
        if n == 13:
            skip = True
            continue
        if skip:
            skip = False
            continue
        total += n
    return total
```

Start coding or generate with AI.

HomeWork

```
def no23(nums):
    return 2 not in nums and 3 not in nums
def make_last(nums):
    return [0]*(2*len(nums)-1) + [nums[-1]]
def double23(nums):
    return nums.count(2) == 2 or nums.count(3) == 2
def fix23(nums):
    if nums[0] == 2 and nums[1] == 3:
        nums[1] = 0
    if nums[1] == 2 and nums[2] == 3:
        nums[2] = 0
    return nums
def start1(a, b):
    count = 0
    if len(a) > 0 and a[0] == 1:
        count += 1
    if len(b) > 0 and b[0] == 1:
        count += 1
    return count
def bigger_two(a, b):
    return a if sum(a) >= sum(b) else b
def make_middle(nums):
    mid = len(nums)//2
    return [nums[mid-1], nums[mid]]
def plus_two(a, b):
    return a + b
def only14(nums):
    for n in nums:
        if n != 1 and n != 4:
            return False
    return True
def fizz_array2(n):
    return [str(i) for i in range(n)]
def no14(nums):
    return not (1 in nums and 4 in nums)
def is_everywhere(nums, val):
    for i in range(len(nums)-1):
        if nums[i] != val and nums[i+1] != val:
            return False
    return True
def either24(nums):
    has22 = False
    has44 = False
    for i in range(len(nums)-1):
        if nums[i]==2 and nums[i+1]==2:
            has22 = True
        if nums[i]==4 and nums[i+1]==4:
            has44 = True
    return has22 ^ has44
def match_up(a, b):
    count = 0
    for i in range(len(a)):
        if a[i] != b[i] and abs(a[i]-b[i]) <= 2:
            count += 1
    return count
def not_alone(nums, val):
    for i in range(1, len(nums)-1):
```

```
        if nums[i]==val and nums[i-1]!=val and nums[i+1]!=val:
            nums[i] = max(nums[i-1], nums[i+1])
    return nums
def zero_max(nums):
    for i in range(len(nums)):
        if nums[i]==0:
            maxodd = 0
            for j in range(i+1, len(nums)):
                if nums[j]%2==1:
                    maxodd = max(maxodd, nums[j])
            nums[i]=maxodd
    return nums
def fix34(nums):
    j = 0
    for i in range(len(nums)):
        if nums[i]==3:
            while nums[j]!=4 or (j>0 and nums[j-1]==3):
                j+=1
            nums[i+1], nums[j] = nums[j], nums[i+1]
    return nums
def fizz_buzz(start, end):
    res=[]
    for i in range(start,end):
        if i%15==0:
            res.append("FizzBuzz")
        elif i%3==0:
            res.append("Fizz")
        elif i%5==0:
            res.append("Buzz")
        else:
            res.append(str(i))
    return res
def array_count9(nums):
    count = 0
    for n in nums:
        if n == 9:
            count += 1
    return count
def array_front9(nums):
    for i in range(min(4, len(nums))):
        if nums[i] == 9:
            return True
    return False
def array123(nums):
    for i in range(len(nums)-2):
        if nums[i]==1 and nums[i+1]==2 and nums[i+2]==3:
            return True
    return False
def string_match(a, b):
    count = 0
    for i in range(min(len(a), len(b))-1):
        if a[i:i+2] == b[i:i+2]:
            count += 1
    return count
def cat_dog(str):
    return str.count("cat") == str.count("dog")
def count_code(str):
    count = 0
    for i in range(len(str)-3):
        if str[i:i+2]=="co" and str[i+3]=="e":
            count += 1
    return count
def end_other(a, b):
    a = a.lower()
    b = b.lower()
    return a.endswith(b) or b.endswith(a)
def xyz_there(str):
    for i in range(len(str)-2):
        if str[i:i+3]=="xyz" and (i==0 or str[i-1]!="."):
            return True
    return False
```

DAY 3 WEDNESDAY

Double-click (or enter) to edit

```
#functions -set of instruction that can be reused without writing the code again
def add():
  a=int(input("enter a number"))
```

```
  b=int(input("enter a number"))
  print(a+b)
for i in range(2):
  add()
```

```
enter a number8
enter a number6
14
enter a number8
enter a number4
12
```

```
def sub():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a-b)
for i in range(2):
  sub()
```

```
enter a number6
enter a number3
3
enter a number9
enter a number2
7
```

```
def multiply():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a*b)
for i in range(2):
  multiply()
```

```
enter a number7
enter a number4
28
enter a number11
enter a number1
11
```

```
def add():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a+b)
def sub():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a-b)
def multiply():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a*b)
def division():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a/b)
def power():
    a=int(input("enter a number"))
    b=int(input("enter a number"))
    print(a**b)
while True:
  c = input("Add/Subtract/Multiply/Division/Power")
  if c.lower() == 'add':
    add()
  elif c.lower()== 'subtract':
    sub()
  elif c.lower()== 'multiply':
    multiply()
  elif c.lower()== 'division':
    division()
  else:
    power()
```

```
4
16
5764801
0
```

```
def add():
  list=[]
  sum=0
  l=int(input("enter the length of the list"))
  for i in range(l):
    a=int(input("Enter a number"))
    list.append(value)
    sum=sum+list[i]
    print(list)
    print(sum)
def sub():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a-b)
def multiply():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a*b)
def division():
  a=int(input("enter a number"))
  b=int(input("enter a number"))
  print(a/b)
def power():
    a=int(input("enter a number"))
    b=int(input("enter a number"))
    print(a**b)
while True:
  c = input("Add/Subtract/Multiply/Division/Power")
  if c.lower() == 'add':
    add()
  elif c.lower()== 'subtract':
    sub()
  elif c.lower()== 'multiply':
    multiply()
  elif c.lower()== 'division':
    division()
  else:
    power()
```

Day 4 Thursday

```
#dictionary =to store data in the form of key and value pairs
person={
    'name':'Yashitha',
    'age': 21,
    'gender':'female'

}
for i in range(5):
  print(person)
```

```
{'name': 'Yashitha', 'age': 21, 'gender': 'female'}
{'name': 'Yashitha', 'age': 21, 'gender': 'female'}
{'name': 'Yashitha', 'age': 21, 'gender': 'female'}
{'name': 'Yashitha', 'age': 21, 'gender': 'female'}
{'name': 'Yashitha', 'age': 21, 'gender': 'female'}
```

```
student_data={
    'name':'Yashitha',
    'college':'Engg College',
    'dob':28/7/2004,
    'phone':787878787787,
    'year':4,
    'college_id':18,


}
print(student_data)
print(student_data['name'])
print(student_data.values())
print(student_data.keys())
print(student_data.items())
```

```
Original student_data: {'name': 'Yashitha', 'college': 'Engg College', 'dob': 0.001996007984031936, 'phone': 787878787787, '
Name: Yashitha
Values: dict_values(['Yashitha', 'Engg College', 0.001996007984031936, 787878787787, 4, 18])
Keys: dict_keys(['name', 'college', 'dob', 'phone', 'year', 'college_id'])
Items: dict_items([('name', 'Yashitha'), ('college', 'Engg College'), ('dob', 0.001996007984031936), ('phone', 787878787787)
```

```
Looping through keys:
name Yashitha
college Engg College
dob 0.001996007984031936
phone 787878787787
year 4
college_id 18

Looping through values:
Yashitha
Engg College
0.001996007984031936
787878787787
4
18

Looping through items (key, value):
name: Yashitha
college: Engg College
dob: 0.001996007984031936
phone: 787878787787
year: 4
college_id: 18
```

```
for key, value in student_data.items():
    print(f"{key}: {value}")
```

```
name: Yashitha
college: Engg College
dob: 0.001996007984031936
phone: 787878787787
year: 4
college_id: 18
```

```
def phonebook_dict():
    phonebook = {'person1': 891723,
                 'person2': 8489498,
                 'person3': 345789}
    search = input("Enter name: ")

    if search in phonebook:
        print(phonebook[search])
    else:
        print("not found")

phonebook_dict()
```

```
Enter name: person1
891723
```

```
text='apple banana apple apple'
words=text.split()
count={}
for i in words:
  count[i]=count.get(i,0)+1
  print(count)
```

```
{'apple': 1}
{'apple': 1, 'banana': 1}
{'apple': 2, 'banana': 1}
{'apple': 3, 'banana': 1}
```

```
#nested dictionary
students={
    'student1':{'name':"a",'age':98,'class':7},
    'student2':{'name':"b",'age':99,'class':8},
    'student3':{'name':"c",'age':97,'class':9},
}
print(students)
```

```
{'student1': {'name': 'a', 'age': 98, 'class': 7}, 'student2': {'name': 'b', 'age': 99, 'class': 8}, 'student3': {'name': 'c
```

DAY 5 Friday

```
mylist = [3, 7, 2, 9, 5, 1, 8, 4, 6]

num = int(input("enter a number to be searched: "))

for index, value in enumerate(mylist):
    if value == num:
        print("Found at index", index)
```

```
        break
else:
    print("Not found!")
```

```
enter a number to be searched: 7
Found at index 1
```

```
from types import new_class
#sorting
mylist = [3, 7, 2, 9, 5, 1, 8, 4, 6]
print(mylist.sort())
new_class=sorted(mylist,reverse=True)
print(new_class)
```

```
None
[9, 8, 7, 6, 5, 4, 3, 2, 1]
```

```
mylist = [3, 7, 2, 9, 5, 1]

n = len(mylist)

for i in range(n-1):
    for j in range(n-1-i):
        if mylist[j] > mylist[j+1]:
            # swap
            temp = mylist[j]
            mylist[j] = mylist[j+1]
            mylist[j+1] = temp

            print("List now:", mylist)

print("\nSorted list:", mylist)
```

```
List now: [3, 2, 7, 9, 5, 1]
List now: [3, 2, 7, 5, 9, 1]
List now: [3, 2, 7, 5, 1, 9]
List now: [2, 3, 7, 5, 1, 9]
List now: [2, 3, 5, 7, 1, 9]
List now: [2, 3, 5, 1, 7, 9]
List now: [2, 3, 1, 5, 7, 9]
List now: [2, 1, 3, 5, 7, 9]
List now: [1, 2, 3, 5, 7, 9]

Sorted list: [1, 2, 3, 5, 7, 9]
```

```
mylist = [1, 2, 3, 4, 5, 6, 7, 8, 9]

num = int(input("Enter number to search: "))

low = 0
high = len(mylist) - 1
found = False

while low <= high:
    mid = (low + high) // 2
    if mylist[mid] == num:
        found = True
        break
    elif num < mylist[mid]:
        high = mid - 1
    else:
        low = mid + 1

if found:
    print("Found at index", mid)
else:
    print("Not found")
```

```
Enter number to search: 1
Found at index 0
```

Homework

```
nums = list(map(int, input("Enter numbers separated by commas: ").split(',')))

length = len(nums)

if length >= 5:
```

```
    fifth_element = nums[4]
    count = nums.count(fifth_element)

    if length == 8 and count == 3:
        print(True)
    else:
        print(False)
else:
    print("List has less than 5 elements")
```

```
Enter numbers separated by commas: 19, 19, 5, 15, 5, 5, 1, 2
True
```

Day 6 Saturday

```
#model training
import numpy as np
from sklearn.linear_model import LinearRegression

hours=np.array([1,2,6,4,3,6,7,3,2,6,1]).reshape(11,1)
marks=np.array([30,35,88,70,50,88,95,50,35,88,30])
print(hours)
print(marks)

model=LinearRegression()
model.fit(hours,marks)
user_input=int(input("Enter the number of hours studied: "))
if user_input<=24 and output<=100:
  output=model.predict([[user_input]])
  print("Marks you will get acc",output)
else:
  print("Invalid input enter between 0 and 24")
```

```
[[1]
 [2]
 [6]
 [4]
 [3]
 [6]
 [7]
 [3]
 [2]
 [6]
 [1]]
[30 35 88 70 50 88 95 50 35 88 30]
Enter the number of hours studied: 1
Marks you will get acc [27.49056604]
```

```
# model training
import numpy as np
from sklearn.linear_model import LinearRegression

age = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 14, 16]).reshape(-1, 1)
height = np.array([75, 85, 92, 98, 105, 112, 118, 125, 131, 138, 150, 160, 168])

print("Ages:\n", age)
print("Heights:\n", height)

model = LinearRegression()
model.fit(age, height)

user_input = float(input("Enter age (must be greater than 0): "))

if user_input > 0:
    output = model.predict([[user_input]])
    print("Predicted height:", round(output[0], 2), "cm")
else:
    print("Invalid input! Age must be greater than 0.")
```

```
Ages:
 [[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
 [10]
 [12]
```

```
 [14]
 [16]]
Heights:
 [ 75  85  92  98 105 112 118 125 131 138 150 160 168]
Enter age (must be greater than 0): 3
Predicted height: 91.92 cm
```

Homework

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([
    [2,2],
    [3,4],
    [5,5],
    [6,7],
    [7,8],
    [9,9]
])

y = np.array([0,0,0,1,1,1])

model = LinearRegression()
model.fit(X,y)

energy = float(input("Enter energy level (1-10): "))
happiness = float(input("Enter happiness level (1-10): "))

prediction = model.predict([[energy,happiness]])

print("Raw prediction:", prediction[0])

if prediction >= 0.5:
    print(" Recommend PARTY music")
else:
    print("Recommend SAD music")
```

```
Enter energy level (1-10): 5
Enter happiness level (1-10): 9
Raw prediction: 1.6066666666666674
 Recommend PARTY music
```

```
import numpy as np
from sklearn.linear_model import LogisticRegression

X = np.array([[2],[3],[4],[8],[10],[12]])

y = np.array([0,0,0,1,1,1])

model = LogisticRegression()
model.fit(X,y)

weight = float(input("Enter animal weight (kg): "))
prediction = model.predict([[weight]])

if prediction[0] == 0:
    print(" It is a CAT")
else:
    print(" It is a DOG")
```

```
Enter animal weight (kg): 4
 It is a CAT
```

Start coding or generate with AI.